

Lecture 14: Binary Search in Array & Three Problems :

 Topic: Searching Smartly using Divide & Decide

Searching ka Real Problem (WHY Binary Search?)

Socho tumhare paas **1,00,000 elements** ka array hai aur tumhe sirf **ek value** dhoondhni hai.

Linear Search

- ek-ek element check
- worst case me 1,00,000 comparisons
- large data me slow 🐢

Binary Search

- beech se check
- aadha data discard
- phir aadha... phir aadha

Lesson

Jab data zyada ho,
tab mehnat nahi — **strategy** chahiye.

What is Binary Search?

 **Binary Search** ek searching technique hai jisme:

- array **sorted** hota hai
- har step par array **2 parts** me divide hota hai
- decide kiya jaata hai:
 - left jaana hai ya
 - right jaana hai

💡 Real-Life Analogy (Dictionary 📖)

- Word dhoondhne ke liye page 1 se start nahi karte
- Beech ka page kholte ho
- Galat half ko ignore kar dete ho

👉 Yehi soch **Binary Search** hai.

⚠️ Binary Search Kab Lagta Hai? (MOST IMPORTANT)

Binary Search **sirf tab** kaam karta hai jab:

- ✓ Data **sorted** ho
- ✓ Decision **monotonic** ho
(matlab ek side hamesha chhoti values, doosri side badi)

✗ Unsorted array par binary search = **logic galat**

📌 Golden Formula

Sorted Data + Left/Right Decision = Binary Search

🧠 Binary Search ka Thinking Framework

Har binary search problem me khud se ye steps poocho:

- 1 Search space kya hai? (**start**, **end**)
- 2 **mid** kaise nikaalna hai?
- 3 **mid** par decision kya hai?
- 4 Kaunsa half discard hoga?
- 5 Loop kab rukega?

👉 Agar ye clear hai, code apne-aap ban jaata hai.

1 2 3 4 Problem 1: First & Last Occurrence of Target

🎯 Problem Statement

Ek **sorted array** diya hai jisme duplicates ho sakte hain.
Tumhe nikalna hai:

- target **pehli baar** kaha aaya

- target **last baar** kaha aaya

Agar target exist nahi karta:

`[-1, -1]`

Example

`nums = [5, 7, 7, 8, 8, 10]`

`target = 8`

`Output = [3, 4]`

? Normal Binary Search Yahan Kyu Fail Hota Hai?

Normal binary search:

- jaise hi target mila → ruk jaata hai ❌

Par yahan:

- hume **boundary** chahiye
- sirf ek index kaam ka nahi

👉 Solution:

- **2 Binary Searches**
 - ek left-biased (first)
 - ek right-biased (last)
-

◆ First Occurrence ka Logic

Agar `arr[mid] == target`:

- mid **answer ho sakta hai**

- par ho sakta hai target aur left me ho

📌 Decision:

Equal → Left side explore karo

◆ Last Occurrence ka Logic

Same thinking, bas direction ulta:

📌 Decision:

Equal → Right side explore karo

🧠 Visual Walkthrough

nums = [5, 7, 7, 8, 8, 10]

index = 0 1 2 3 4 5

target = 8

First Occurrence

- mid = 2 → 7 < 8 → right
- mid = 4 → equal → save → left
- mid = 3 → equal → save → left
👉 first = 3

Last Occurrence

- mid = 2 → right
 - mid = 4 → equal → save → right
 - mid = 5 → 10 > 8 → left
👉 last = 4
-

Code: First & Last Occurrence :

```
vector<int> searchRange(vector<int>& nums, int target) {  
  
    int n = nums.size();  
  
    // ----- FIRST OCCURRENCE -----  
  
    int start = 0;  
  
    int end = n - 1;  
  
    int first = -1;    // default: target not found  
  
    while (start <= end) {  
  
        // Mid index (overflow safe)  
        int mid = start + (end - start) / 2;  
  
        if (nums[mid] == target) {  
            first = mid;    // possible answer store  
            end = mid - 1;  // left side me aur ho sakta hai  
        }  
        else if (nums[mid] < target) {  
            start = mid + 1;    // target right side me hoga  
        }  
        else {  
            end = mid - 1;    // target left side me hoga  
        }  
    }  
  
    // ----- LAST OCCURRENCE -----
```

```
start = 0;

end = n - 1;

int last = -1;

while (start <= end) {

    int mid = start + (end - start) / 2;

    if (nums[mid] == target) {
        last = mid;          // possible answer store
        start = mid + 1;    // right side me aur ho sakta hai
    }

    else if (nums[mid] < target) {
        start = mid + 1;
    }

    else {
        end = mid - 1;
    }
}

return {first, last};
}
```

🕒 Time & Space Complexity

- **Time Complexity:** $O(\log n)$
(har step me search space aadha hota jaata hai)

- **Space Complexity:** $O(1)$
(sirf variables, koi extra memory nahi)
-

Problem 2: Search Insert Position

Problem Statement

Ek **sorted array** diya hai aur ek **target**.

- agar target mil jaaye → uska index
 - agar na mile → wo index jahan insert karna chahiye (taaki array sorted rahe)
-

Example

`nums = [1, 3, 5, 6]`

`target = 2`

`Output = 1`

Core Insight (CONFUSION KILLER)

 Jab binary search **fail** hota hai:

- **start** pointer exactly us position par hota hai jahan element insert hona chahiye

 Failure bhi **information** deta hai.

Code: Search Insert Position :

```
int searchInsert(vector<int>& nums, int target) {
```

```
    int start = 0;
```

```
    int end = nums.size() - 1;
```

```
int index = nums.size();    // worst case: last position

while (start <= end) {

    int mid = start + (end - start) / 2;

    if (nums[mid] == target) {
        return mid;        // exact match mil gaya
    }
    else if (nums[mid] < target) {
        start = mid + 1;    // right half me jao
    }
    else {
        index = mid;        // possible insert position
        end = mid - 1;      // left half explore karo
    }
}

return index;
}
```

Problem 3: Sqrt(x)

Problem Statement

Ek non-negative integer x diya hai.

Tumhe return karna hai **floor value of $\sqrt{x}$** .

? sqrt(x) me Binary Search Kyu Lagta Hai?

Socho:

- $\sqrt{x}$ ka answer **0 se x** ke beech hota hai
- agar `mid * mid <= x` → mid possible answer
- jaise-jaise mid badhega → square bhi badhega

👉 Behaviour **monotonic**

👉 Binary Search perfectly fit

💻 Code: sqrt(x) using Binary Search :

```
int mySqrt(int x) {  
  
    int start = 0;  
  
    int end = x;  
  
    int ans = 0;    // floor sqrt store karega  
  
    while (start <= end) {  
  
        long long mid = start + (end - start) / 2;  
  
        // Agar perfect square mil gaya  
        if (mid * mid == x) {  
            return mid;  
        }  
  
        // Agar square chhota hai, to mid possible answer  
        else if (mid * mid < x) {  
            ans = mid;    // store current answer  
            start = mid + 1; // bada number try karo
```

```

    }

    // Agar square zyada ho gaya

    else {

        end = mid - 1;    // chhota number try karo

    }

}

return ans;    // floor value return
}

```

- **Time:** $O(\log x)$
- **Space:** $O(1)$

Coding ≠ Syntax (REAL GYAAN)

- ✗ Coding = language ratna
- ✗ Coding = syntax yaad karna

✓ Coding =

- problem ko todna
- pattern samajhna
- sahi approach choose karna

🚀 **DSA = Thinking Skill,**
jo Web Dev, Backend, System Design — sab jagah kaam aati hai.

🌟 **FINAL QUICK SUMMARY**

- ✓ Binary Search = Divide & Decide
- ✓ First/Last Occurrence = Boundary search
- ✓ Search Insert = Failure gives position
- ✓ $\text{sqrt}(x)$ = Binary Search on answer
- ✓ Time = $O(\log n)$
- ✓ Space = $O(1)$
- ✓ Coding = Logic + Common Sense